`mdarray`: An Owning Multidimensional Array Analog of `mdspan`

Document #:	P1684R4
Date:	2023-01-14
Project:	Programming Language C++ Library Evolution

Reply-to:	Christian Trott <crtrott@sandia.gov> Daisy Hollman <me@dsh.fyi> Mark Hoemmen <mark.hoemmen@gmail.com> Daniel Sunderland <dansunderland@gmail.com> Damien Lebrun-Grandie <lebrungrandt@ornl.gov>

Contents

§ 1 Revision History

§ 1.1 P1684R3: 2022-07 Mailing

§ 1.1.0.1 Changes from R3

— drop the “size constructible container” requirements and simply use preconditions on relevant constructors

§ 1.1.0.2 Changes from R2

— bring in line with P0009 and follow up papers P2554, P2553, P2599 and P2406
— add new container requirements: *size constructible containers* (bikeshedding needed)
— add deduction guide for `mdspan` from `mdarray`
— remove constructors from `ranges`/`initializer_list` etc. (can be done via moving container in)
  — waiting for feedback from other sgs to see which convenience constructors we should have
— add deduction guides
— update explanatory wording

§ 1.2 P1684r2: 2022-04 Mailing

§ 1.2.0.1 Changes from R1

— update and reword non-wording text to harmonize with P0009R16
— fix synopsis missing Alloc argument in some places
— fix constraints on some constructors to allow `std::array` as container

— add missing wording for constructors from `std::initializer_list`

— add constructors from ranges

— always use `std::vector` as the default container type, based on LEWG guidance

— remove container accessor function

  — it's a bad idea to allow someone to resize the container owned by an `mdarray` …

— add new `mdarray` constructors from `mdspan`

— add `view` member function; it returns an `mdspan` that views the `mdarray`'s elements

1.3 P1684r1: 2022-03 Mailing

1.3.0.1 Changes from R0

— harmonize with P0009R15

— don't use ContainerPolicy, simply use Container as fourth template argument

— added wording

2 Motivation

[**P0009R18?**] proposed `mdspan`, a nonowning multidimensional array abstraction. It was voted into the C++23 draft. This proposal builds on `mdspan` by introducing `mdarray`, an *owning* multidimensional array that interoperates with `mdspan`. The `mdarray` class is to `vector` as `mdspan` is to `span`. Owning semantics can make it easier for users to express common cases, like returning an array from a function. It also makes it much easier to create a multi dimensional array for use:

C++23:

```
// Create a mapping so one knows how many
// elements the buffer needs to have
layout_right::mapping<dextents<2>> map(N,M);

// Create the underlying data object
vector<double> buffer(map.required_span_size());

// Create the mdspan

mdspan matrix(buffer.data(), N, M);
```

This Work:

```
mdarray<double, dextents<3,3>> matrix(N,M);
```

In addition, especially for arrays with small, compile-time extents, `mdarray`'s owning semantics make fewer demands on interprocedural analysis for optimization. In particular,

the lack of indirection due to `mdarray` owning its data makes it easier for compilers to deduce that the data could be stored in registers.

§ 3 Design

One major goal of the design for `mdarray` is to parallel the design of `mdspan` as much as possible, with the goals of reducing cognitive load for users already familiar with `mdspan` and of incorporating the lessons learned from over a decade of experience with [**P0009R18?**] and libraries of similar design. This paper assumes the reader has read and is already familiar with [**P0009R18?**].

§ 3.1 Design overview

The analogy to `mdspan` can be seen in the declaration of the proposed design for `mdarray`.

```
template<class ElementType,
         class Extents,
         class LayoutPolicy = layout_right,
         class Container = see-below>
  class mdarray;
```

This intentionally parallels the design of `mdspan` in [**P0009R18?**], which has the following signature.

```
template<class ElementType,
         class Extents,
         class LayoutPolicy = layout_right,
         class AccessorPolicy = default_accessor<ElementType>>
  class mdspan;
```

Our original `mdarray` proposal [P1684R0] had a `ContainerPolicy` instead of a `Container` template parameter. The `ContainerPolicy` provided functionality like that of `mdspan`'s `AccessorPolicy`, and would also select the actual container type used to store the `mdarray`'s data.

For the current revision of this proposal, we have decided that the complexity of a `ContainerPolicy` is not actually required for `mdarray`. The vast majority of cases where customization of the `AccessorPolicy` is required to modify the access behavior, are local contextual requirements that are better served by `mdspan`. For example, one might have code that creates and uses an `mdarray` in several different ways. One loop over the array might have write conflicts that an atomic accessor would resolve. Changing how one views data in a way limited to a particular context is the job of a view. That is, in this case, it would make the most sense to create a temporary `mdspan` with an atomic accessor that views the original `mdarray`, for use in the particular loop that needs atomic access.

§ 3.2 Differences between `mdarray` and `mdspan`

By design, `mdarray` is as similar as possible to `mdspan`, except with container semantics instead of reference semantics. However, the use of container semantics calls for a few differences.

§ 3.2.1 Deep `constness`

The most notable difference from `mdspan` is deep `constness`. Like all reference semantic types in the standard, `mdspan` has shallow `constness`, but container types in the standard library propagate `const` through their access functions. Thus, `mdarray` needs `const` and non-`const` versions of every analogous operation in `mdspan` that interacts with the underlying data.

```
template<class ElementType, class Extents, class LayoutPolicy, class
        Container>
class mdarray {
  /* ... */

  // also in mdspan:
  using pointer = /* ... */;
  // only in mdarray:
  using const_pointer = /* ... */;

  // also in mdspan:
  using reference = /* ... */;
  // only in mdarray:
  using const_reference = /* ... */;

  // analogous to mdspan, except with const_reference return type:
  template<class... IndexType>
    constexpr const_reference operator[](IndexType...) const;
  template<class IndexType, size_t N>
    constexpr const_reference operator[](const array<IndexType, N>&)
        const;
  // non-const overloads only in mdarray:
  template<class... IndexType>
    constexpr reference operator[](IndexType...);
  template<class IndexType, size_t N>
    constexpr reference operator[](const array<IndexType, N>&);

  // also in mdspan, except with const_pointer return type:
  constexpr const_pointer data() const noexcept;
  // non-const overload only in mdarray:
  constexpr pointer data() noexcept;

  /* ... */
};
```

§ 3.2.2 Interoperability between `mdarray` and `mdspan`

The `mdarray` class needs a means of interoperating with `mdspan` in roughly the same way as contiguous containers interact with `span`, or as `string` interacts with `string_view`. One way we could do this would be by adding a constructor to `mdspan`. This which would be more consistent with the analogous features in `span` and `string_view`. However, in the

interest of avoiding modifications to an in-flight proposal, we instead propose using a member function of `mdarray` for this functionality for now. This member function is tentatively named `to_mdspan()`, but we welcome suggestions for other names.

One advantage of this member function is that it could serve as a starting point for a general customization point in C++ for obtaining an `mdspan` generically. `to_mdspan` would effectively work like `begin` and other such customization points, which allow C++ facilities to interact with user provided classes.

We would be happy to change this based on design direction from LEWG.

```
template<class ElementType, class Extents, class LayoutPolicy, class
        Container>
class mdarray {
  /* ... */

  // only in mdarray:
  using mdspan_type = /* ... */;
  using const_mdspan_type = /* ... */;

  template<class OT, class OE, class OL, class OA>
    operator mdspan() const;

  template<class OtherAccessorType = default_accessor<element_type>>
    auto to_mdspan(const OtherAccessorType& a =
                     default_accessor<element_type>()) const noexcept;
  template<class OtherAccessorType = default_accessor<const element_type>>
    auto to_mdspan(const OtherAccessorType& a =
                     default_accessor<const element_type>()) const
        noexcept;

  /* ... */
};
```

§ 3.2.3 `Container` instead of `AccessorPolicy`

As we discuss elsewhere, the `mdspan` class has an `AccessorPolicy` template parameter, while `mdarray` has a `Container` template parameter.

```
template<class ElementType, class Extents, class LayoutPolicy, class
        Container>
class mdarray {
  /* ... */

  // only in mdarray
 using container_type = Container;

  /* ... */
};
```

§ 3.2.4 Constructors and assignment operators

The constructors and assignment operators of `mdspan` and `mdarray` have a few differences. The `mdspan` class provides a compatible `mdspan` copy-like constructor and copy-like assignment operator, with proper constraints and expectations to enforce compatibility of shape, layout, and size. Since `mdarray` has owning semantics, we also need move-like versions of these:

```
template<class ElementType, class Extents, class LayoutPolicy, class
        ContainerPolicy>
class mdarray {
  /* ... */

  // analogous to mdspan:
  template<class ET, class Exts, class LP, class CP>
    constexpr mdarray(const mdarray<ET, Exts, LP, CP>&);
  template<class ET, class Exts, class LP, class CP>
    constexpr mdarray& operator=(const mdarray<ET, Exts, LP, CP>&);
  // only in mdarray:
  template<class ET, class Exts, class LP, class CP>
    constexpr mdarray(mdarray<ET, Exts, LP, CP>&&) noexcept(see-below);
  template<class ET, class Exts, class LP, class CP>
    constexpr mdarray& operator=(mdarray<ET, Exts, LP, CP>&&) noexcept;

  /* ... */
};
```

(The `noexcept` clauses on these constructors and operators should probably actually derive from `noexcept` clauses on the analogous functionality for the element type and policy types.)

Additionally, the analog of the `mdspan(pointer, IndexType...)` constructor for `mdarray` does not need a `pointer` argument, since the `mdarray` owns the data and thus should be able to construct it from sizes:

```
template<class ElementType, class Extents, class LayoutPolicy, class
        ContainerPolicy>
class mdarray {
  /* ... */

  // only in mdarray
  template <class... IndexType>
  explicit constexpr mdarray(IndexType...);

  /* ... */
};
```

Note that in the completely static extents case, this is ambiguous with the default constructor. For consistency in generic code, the semantics of this constructor should be preferred over those of the default constructor in that case.

By this same logic, we arrive at the `mapping_type` constructor analogs:

```
template<class ElementType, class Extents, class LayoutPolicy, class
        Container>
class mdarray {
  /* ... */

  // only in mdarray
  explicit constexpr mdarray(const mapping_type&);

  /* ... */
};
```

There is some question as to whether we should also have constructors that take `container_type` instances in addition to indices. Consistency with standard container adapters like `std::priority_queue` would dictate that we should. Note that these constructors would deep-copy the input container.

```
template<class ElementType, class Extents, class LayoutPolicy, class
        Container>
class mdarray {
  /* ... */

  // only in mdarray
  explicit constexpr mdarray(const mapping_type&);
  constexpr mdarray(const container_type&, const mapping_type&);

  /* ... */
};
```

Moreover, containers such as `vector` and container adapters like `queue` have a significant number of other constructors which copy the data on input, such as constructors taking ranges, input iterator pairs, and initializer lists. The R2 version of this paper had a number of those, but in combination with allocator arguments it lead to an explosion of constructors. A draft version seen by LEWG on 2022-07-12 sported 41 constructors. However almost all these constructors would simply forward arguments to the constructor of the container owned by `mdarray`. LEWG provided feedback that we should consider reducing the amount of constructors, since at a minimum one could inline construct the `container` itself, and move it into the `mdarray` upon construction.

```
// with convenience constructors:
mdarray a(first, last, N, M);

// without
mdarray a(std::move(vector(first, last)), N, M);
```

In R3 we decided to radically cut back on constructors and simply leave those out.

Finally, `mdarray` does not have the analog of `mdspan`'s constructor that takes an `array<IndexType, N>` of dynamic extents. This avoids any ambiguity or confusion with a constructor that takes a container instance, in the case where the `container` happens to be an `array<IndexType, N>`.

§ 3.3 `Extents` design reused

As with `mdspan`, the `Extents` template parameter to `mdarray` shall be a template instantiation of `std::extents`, as described in [**P0009R18?**]. The concerns addressed by this aspect of the design are exactly the same in `mdarray` and `mdspan`, so using the same form and mechanism seems like the right thing to do here.

§ 3.4 `LayoutPolicy` design reused

While not quite as straightforward, the decision to use the same design for `LayoutPolicy` from `mdspan` in `mdarray` is still quite obviously the best choice. The only pieces are perhaps a less perfect fit are the `is_contiguous()` and `is_always_contiguous()` requirements. While noncontiguous use cases for `mdspan` are quite common (e.g., `subspan()`), noncontiguous use cases for `mdarray` are expected to be a bit more arcane. Nonetheless, reasonable use cases do exist (for instance, padding of the fast-running dimension in anticipation of a resize operation), and the reduction in cognitive load due to concept reuse certainly justifies reusing `LayoutPolicy` for `mdarray`.

§ 3.5 `AccessorPolicy` replaced by `Container`

By far the most complicated aspect of the design for `mdarray` is the analog of the `AccessorPolicy` in `mdspan`. The `AccessorPolicy` for `mdspan` is designed for nonowning semantics. It provides a `pointer` type, a `reference` type, and a means of converting from a pointer and an offset to a reference. Beyond the lack of an allocation mechanism (that would be needed by `mdarray`), the `AccessorPolicy` requirements address concerns normally addressed by the allocation mechanism itself. For instance, the C++ named requirements for `Allocator` allow for the provision of the `pointer` type to `std::vector` and other containers. Arguably, consistency between `mdarray` and standard library containers is far more important than with `mdspan` in this respect. Several approaches to addressing this incongruity are discussed below.

§ 3.5.1 Expected behavior of motivating use cases

Regardless of the form of the solution, there are several use cases where we have a clear understanding of how we want them to work. As alluded to above, perhaps *the* most important motivating use case for `mdarray` is that of small, fixed-size extents. Consider a fictitious (not proposed) function, *get-underlying-container*, that somehow retrieves the underlying storage of an `mdarray`. For an `mdarray` of entirely fixed sizes, we would expect the default implementation to return something that is (at the very least) convertible to `array` of the correct size:

```
auto a = mdarray<int, 3, 3>();
std::array<int, 9> data = get-underlying-container(a);
```

(Whether or not a reference to the underlying container should be obtainable is slightly less clear, though we see no reason why this should not be allowed.) The default for an `mdarray` with variable extents is only slightly less clear, though it should almost certainly meet the requirements of *contiguous container* (**[container.requirements.general]**/13).

The default model for *contiguous container* of variable size in the standard library is `vector`, so an entirely reasonable outcome would be to have:

```
auto a = mdarray<int, 3, dynamic_extent>();
std::vector<int> data = get-underlying-container(a);
```

Moreover, taking a view of a `mdarray` should yield an analogous `mdspan` with consistent semantics (except, of course, that the latter is nonowning). We provisionally call the method for taking a view of an `mdarray` "`to_mdspan()`":

```
template <class T, class Extents, class LayoutPolicy, class
        ContainerPolicy>
void frobnicate(mdarray<T, Extents, LayoutPolicy, ContainerPolicy> data)
{
  auto data_view = data.to_mdspan();
  /* ... */
}
```

In order for this to work, `Container::data()` should be required to return `T*`. That way, interoperability with `mdspan` is trivial, since it can simply be created as:

```
template <class T, class Extents, class LayoutPolicy, class Container>
void frobnicate(mdarray<T, Extents, LayoutPolicy, Container> a)
{
  mdspan a_view(a.data(), a.mapping());
  /* ... */
}
```

§ 3.5.2 Analogs in the standard library: Container adapters

Perhaps the best analogs for what `mdarray` is doing with respect to allocation and ownership are the container adaptors (**[container.adaptors]**), since they imbue additional semantics to what is otherwise an ordinary container. These all take a `Container` template parameter, which defaults to `deque` for `stack` and `queue`, and to `vector` for `priority_queue`. The allocation concern is thus delegated to the container concept, reducing the cognitive load associated with the design. While this design approach overconstrains the template parameter slightly (that is, not all of the requirements of the `Container` concept are needed by the container adaptors), the simplicity arising from concept reuse more than justifies the cost of the extra constraints.

It is difficult to say whether the use of `Container` directly, as with the container adaptors, is also the correct approach for `mdarray`. There are pieces of information that may need to be customized in some very reasonable use cases that are not provided by the standard container concept. The most important of these is the ability to produce a semantically consistent `AccessorPolicy` when creating a `mdspan` that refers to a `mdarray`. (Interoperability between `mdspan` and `mdarray` is considered a critical design requirement because of the nearly complete overlap in the set of algorithms that operate on them.) For instance, given a `Container` instance `c` and an `AccessorPolicy` instance `a`, the behavior of `a.access(p, n)` should be consistent with the behavior of `c[n]` for a `mdspan` wrapping `a` that is a view of a `mdarray` wrapping `c` (if `p` is `c.begin()`). But because `c[n]` is part of

the container requirements and thus may encapsulate any arbitrary mapping from an offset of `c.begin()` to a reference, the only reasonable means of preserving these semantics for arbitrary container types is to reference the original container directly in the corresponding `AccessorPolicy`. In other words, the signature for the `view()` method of `mdarray` would need to look something like (ignoring, for the moment, whether the name for the type of the accessor is specified or implementation-defined):

```
template<class ElementType,
         class Extents,
         class LayoutPolicy,
         class Container>
struct mdarray {
 /* ... */
 mdspan<
   ElementType, Extents, LayoutPolicy,
   container_reference_accessor<Container>>
 view() const noexcept;
 /* ... */
};

template <class Container>
struct __container_reference_accessor { // not proposed
  using pointer = Container*;
  /* ... */
  template <class Integer>
  reference access(pointer p, Integer offset) {
    return (*p)[offset];
  }
  /* ... */
};
```

However, this approach comes at the cost of an additional indirection: one for the pointer to the container, and one for the container dereference itself. This is likely unacceptable cost in a facility designed to target performance-sensitive use cases. The situation for the `offset` requirement (which is used by `submdspan`) is potentially even worse for arbitrary non-contiguous containers, adding up to one indirection per invocation of `submdspan`. This is likely unacceptable in many contexts.

Nonetheless, using refinements of the existing `Container` concept directly with `mdarray` is an incredibly attractive option. This is because it avoids the introduction of an extra concept, and thus significantly decreases the cognitive cost of the abstraction. Thus, direct use of the existing `Container` concept hierarchy should be preferred to other options unless the shortcomings of the existing concept are so irreconcilable (or so complicated to reconcile) as to create more cognitive load than is needed for an entirely new concept.

One straightforward way to resolve the above concerns with arbitrary container types, is to simply restrict what type of containers can be used. Specifically, we would restrict it such that creating an `mdspan` with `default_accessor` is straightforward. Thus we would require `decltype(Container::data())` to denote `ElementType*`, and `&c[i]` to equal `c.data() + i` for all `i` in the range of [`0`, `c.size()`).

In what follows, we discuss two design alternatives.

3.5.3 (Not proposed) alternative: A dedicated `ContainerPolicy` concept

Despite the additional cognitive load, there are a few arguments in favor of using a dedicated concept for the container description of `mdarray`. As is often the case with concept-driven design, the implementation of `mdarray` only needs a relatively small subset of the interface elements in the `Container` concept hierarchy. This alone is not enough to justify an additional concept external to the existing hierarchy; however, there are also quite a few features missing from the existing container concept hierarchy, without which an efficient `mdarray` implementation may be difficult or impossible. As alluded to above, conversion to an `AccessorPolicy` for the creation of a `mdspan` is one missing piece. (Another, interestingly, is sized construction of the container mixed with allocator awareness, which is surprisingly lacking in the current hierarchy somehow.) For these reasons, it is worth exploring a design based on analogy to the `AccessorPolicy` concept rather than on analogy to `Container`. If we make that abstraction owning, we might call it something like `_ContainerLikeThing` (not proposed here; included for discussion). In that case, a model of the `_ContainerLikeThing` concept that meets the needs of `mdarray` might look something like:

```
template <class ElementType, class Allocator=std::allocator<ElementType>>
struct vector_container_like_thing // models _ContainerLikeThing
{
public:
  using element_type = ElementType;
  using container_type = std::vector<ElementType, Allocator>;
  using allocator_type = typename container_type::allocator_type;
  using pointer = typename container_type::pointer;
  using const_pointer = typename container_type::const_pointer;
  using reference = typename container_type::reference;
  using const_reference = typename container_type::const_reference;
  using accessor_policy = std::accessor_basic<element_type>;
  using const_accessor_policy = std::accessor_basic<const element_type>;

  // analogous to `access` method in `AccessorPolicy`
  reference access(ptrdiff_t offset) { return __c[size_t(offset)]; }
  const_reference access(ptrdiff_t offset) const { return
        __c[size_t(offset)]; }

  // Interface for mdspan creation
  accessor_policy make_accessor_policy() { return { }; }
  const_accessor_policy make_accessor_policy() const { return { }; }
  typename pointer data() { return  __c.data(); }
  typename const_pointer data() const { return  __c.data(); }

  // Interface for sized construction
  static vector_container_policy create(size_t n) {
    return vector_container_like_thing{container_type(n, element_type{})};
  }
  static vector_container_policy create(size_t n, allocator_type const&
        alloc) {
```

```
    return vector_container_like_thing{container_type(n, element_type{},
        alloc)};
  }

  container_type __c;
};
```

This approach solves many of the problems associated with using the `Container` concept directly. It is the most flexible and provides the best compatibility with `mdspan`, since the conversion to analogous `AccessorPolicy` is fully customizable. This comes at the cost of additional cognitive load, but this can be justified based on the observation that almost half of the functionality in the above sketch is absent from the container hierarchy: the `make_accessor_policy()` requirement and the sized, allocator-aware container creation (`create(n, alloc)`) have no analogs in the container concept hierarchy. Non-allocator-aware creation (`create(n)`) is analogous to sized construction from the sequence container concept, the `data()` method is analogous to `begin()` on the contiguous container concept, and `access(n)` is analogous to `operator[]` or `at(n)` from the optional sequence container requirements. Even for these latter pieces of functionality, though, we are required to combine several different concepts from the `Container` hierarchy. Based on this analysis, we have decided it is reasonable to pursue designs for this customization point that diverge from `Container`, including ones that use `AccessorPolicy` as a starting point. Given a better design, we would definitely consider reversing direction on this decision, but despite significant effort, we were unable to find a design that was more than an awkward and forced fit for the `Container` concept hierarchy.

3.5.4 (Not proposed) alternative: `ContainerPolicy` subsumes `AccessorPolicy`

The above approach has the significant drawback that the `_ContainerLikeThing` is an owning abstraction fairly similar to a container that diverges from the `Container` hierarchy. We initially explored this direction because it avoids having to provide a `mdarray` constructor that takes both a `Container` and a `ContainerPolicy`, which we felt was a “design smell.” Another alternative along these lines is to make the `mdarray` itself own the container instance and have the `ContainerPolicy` (name subject to bikeshedding; maybe `ContainerFactory` or `ContainerAccessor` is more appropriate?) be a nonowning abstraction that describes the container creation and access. While this approach leads to an ugly `mdarray(container_type, mapping_type, ContainerPolicy)` constructor, the analog that constructor affords to the `mdspan(pointer, mapping_type, AccessorPolicy)` constructor is a reasonable argument in favor of this design despite its quirkiness. Furthermore, this approach affords the opportunity to explore a `ContainerPolicy` design that subsumes `AccessorPolicy`, thus providing the needed conversion to `AccessorPolicy` for the analogous `mdspan` by simple subsumption. More importantly, this subsumption would significantly decrease the cognitive load for users already familiar with `mdspan`. A model of `ContainerPolicy` for this sort of approach might look something like:

```
template <class ElementType, class Allocator=std::allocator<ElementType>>
struct vector_container_policy // models ContainerPolicy (and thus
        AccessorPolicy)
{
public:
  using element_type = ElementType;
  using container_type = std::vector<ElementType, Allocator>;
  using allocator_type = typename container_type::allocator_type;
  using pointer = typename container_type::pointer;
  using const_pointer = typename container_type::const_pointer;
  using reference = typename container_type::reference;
  using const_reference = typename container_type::const_reference;
  using offset_policy = vector_container_policy<ElementType, Allocator>

  // ContainerPolicy requirements:
  reference access(container_type& c, ptrdiff_t i) { return c[size_t(i)];
        }
  const_reference access(container_type const& ptrdiff_t i) const { return
        c[size_t(i)]; }

  // ContainerPolicy requirements (interface for sized construction):
  container_type create(size_t n) {
    return container_type(n, element_type{});
  }
  container_type create(size_t n, allocator_type const& alloc) {
    return container_type(n, element_type{}, alloc);
  }

  // AccessorPolicy requirement:
  reference access(pointer p, ptrdiff_t i) { return p[i]; }
  // For the const analog of AccessorPolicy:
  const_reference access(const_pointer p, ptrdiff_t i) const { return
        p[i]; }

  // AccessorPolicy requirement:
  pointer offset(pointer p, ptrdiff_t i) { return p + i; }
  // For the const analog of AccessorPolicy:
  const_pointer offset(const_pointer p, ptrdiff_t i) const { return p + i;
        }

  // AccessorPolicy requirement:
  element_type* decay(pointer p) { return p; }
  // For the const analog of AccessorPolicy:
  const element_type* decay(pointer p) const { return p; }

};
```

The above sketch makes clear the biggest challenge with this approach: the mismatch in shallow versus deep `const`ness in for an abstractions designed to support `mdspan` and `mdarray`, respectively. The `ContainerPolicy` concept thus requires additional `const`-qualified overloads of the basis operations. Moreover, while the `ContainerPolicy` itself can be obtained directly from the corresponding `AccessorPolicy` in the case of the non-`const` method for creating the corresponding `mdspan` (provisionally called `view()`), the `const`-qualified version needs to adapt the policy, since the nested types have the wrong

names (e.g., `const_pointer` should be named `pointer` from the perspective of the `mdspan` that the `const`-qualified `view()` needs to return). This could be fixed without too much mess using an adapter (that does not need to be part of the specification):

```
template <ContainerPolicy P>
class __const_accessor_policy_adapter { // models AccessorPolicy
public:
  using element_type = add_const_t<typename P::element_type>;
  using pointer = typename P::const_pointer;
  using reference = typename P::const_reference;
  using offset_policy = __const_accessor_policy_adapter<typename
        P::offset_policy>;

  reference access(pointer p, ptrdiff_t i) { return acc_.access(p, i); }
  pointer offset(pointer p, ptrdiff_t i) { return acc_.offset(p, i); }
  element_type* decay(pointer p) { return acc_.decay(p); }

private:
  [[no_unique_address]] add_const_t<P> acc_;
};
```

Compared to simply using a `container` as the argument, this approach has the benefit of enabling `mdarray` to use containers for which `data()[i]` is not giving access to the same element as `container[i]`. However, after more consideration we believe that the need for supporting such containers as underlying storage for `mdarray` is likely fairly niche. Furthermore, we believe one could later extent the design of `mdarray` to allow for such ContainerPolicies, even if the initial design only allows for a restricted set of containers.

§ 3.5.5 Extension for accessor policy from container

Most of the drawbacks of using container directly as the template parameter for `mdarray` addressed in the above design alternatives, can likely be remedied by introducing a customization point to obtain an accessor policy and the associated `data_handle` from an existing container. We do not propose such a customization point in this paper, but don't expect this to be a major issue.

§ 3.5.6 Safety and other issues with containers

§ 3.5.6.1 Size and access guarantees for containers

We have to somehow guarantee that containers constructed from sizes (or if such constructors are reintroduced from iterator pairs, initializer lists or ranges), are actually large enough after the construction so we can index into them. However, we don't want full sequence container requirements (also there is no named requirement for a constructor which takes an integral only).

To illustrate the problem consider a user defined container with a static size but that has a constructor which takes an init value.

```
user::static_vector<int, 200> vec(1000);

assert(vec.size()==200);
```

From the perspective of `mdarray` this container looks like its constructible from a size, but that is not actually what the container does.

We previously proposed a new set of named optional requirements for containers which guarantee the desired behavior.

These requirements would need to go into the general container requirements, or be a new named requirement? But maybe there is also another way to require these semantics for containers used with `mdarray` instead, for example one might say it is undefined behavior if a container class is used which does not have a size of `n` when constructed with `n` as its sole argument.

In Revision 4 we removed those named container requirements and instead propose an alternative approach - made easier by the removal of a lot of constructors in R3. Specifically, we enforce that containers constructed from sizes have that expected size via preconditions on the relevant containers.

3.5.6.2 Size requirements for construction from containers etc.

We explicitly require that if an `mdarray` is constructed from some existing set of elements (container, iterators, range, etc.), that the size of that container etc. is larger or equal to the mappings required span size. The larger or equal is intentional, because the mapping may already be not exhaustive. I.e. the `mdarray` may not end up accessing all elements in its own container. But in that case it is not clear why having unused elements at the end should be different from having unused elements somewhere else in the container.

4 Wording

Insert the following after section 24.6.6

24.6.□ Class template `mdarray` [mdarray]

24.6.□.1 `mdarray` overview [mdarray.overview]

1 `mdarray` is a multidimensional array of elements.

```
namespace std {

template<class ElementType, class Extents, class LayoutPolicy, class
        Container =
         vector<ElementType>>
```

```
class mdarray {
public:
  using extents_type = Extents;
  using layout_type = LayoutPolicy;
  using container_type = Container;
  using mapping_type = typename layout_type::template
        mapping<extents_type>;
  using element_type = ElementType;
  using mdspan_type = mdspan<element_type, extents_type, layout_type>;
  using const_mdspan_type = mdspan<const element_type, extents_type,
        layout_type>;
  using value_type = element_type;
  using index_type = typename Extents::index_type;
  using size_type = typename Extents::size_type;
  using rank_type = typename Extents::rank_type;
  using pointer = typename container_type::pointer;
  using reference = typename container_type::reference;
  using const_pointer = typename container_type::const_pointer;
  using const_reference = typename container_type::const_reference;

  static constexpr rank_type rank() noexcept { return
        extents_type::rank(); }
  static constexpr rank_type rank_dynamic() noexcept { return
        extents_type::rank_dynamic(); }
  static constexpr size_t static_extent(rank_type r) noexcept
    { return extents_type::static_extent(r); }
  constexpr index_type extent(rank_type r) const noexcept { return
        extents().extent(r); }

  // [mdarray.ctors], mdarray constructors
  constexpr mdarray() requires(rank_dynamic() != 0) = default;
  constexpr mdarray(const mdarray& rhs) = default;
  constexpr mdarray(mdarray&& rhs) = default;

  template<class... OtherIndexTypes>
    explicit constexpr mdarray(OtherIndexTypes... exts);
  explicit constexpr mdarray(const extents_type& ext);
  explicit constexpr mdarray(const mapping_type& m);

  constexpr mdarray(const extents_type& ext, const value_type& val);
  constexpr mdarray(const mapping_type& m, const value_type& val);

  template<class... OtherIndexTypes>
    constexpr mdarray(const container_type& c, OtherIndexTypes... exts);
  constexpr mdarray(const container_type& c, const extents_type& ext);
  constexpr mdarray(const container_type& c, const mapping_type& m);

  template<class... OtherIndexTypes>
    constexpr mdarray(container_type&& c, OtherIndexTypes... exts);
  constexpr mdarray(container_type&& c, const extents_type& ext);
  constexpr mdarray(container_type&& c, const mapping_type& m);


  template<class OtherElementType, class OtherExtents,
           class OtherLayoutPolicy, class OtherContainer>
    explicit(see below)
```

```
  constexpr mdarray(
    const mdarray<OtherElementType, OtherExtents,
                  OtherLayoutPolicy, OtherContainer>& other);

template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class Accessor>
  explicit(see below)
  constexpr mdarray(const mdspan<OtherElementType, OtherExtents,
                                 OtherLayoutPolicy, Accessor>& other);

// [mdarray.ctors.alloc], mdarray constructors with allocators
template<class Alloc>
  constexpr mdarray(const extents_type& ext, const Alloc& a);
template<class Alloc>
  constexpr mdarray(const mapping_type& m, const Alloc& a);

template<class Alloc>
  constexpr mdarray(const extents_type& ext, const value_type& val,
       const Alloc& a);
template<class Alloc>
  constexpr mdarray(const mapping_type& m, const value_type& val, const
       Alloc& a);

template<class Alloc>
  constexpr mdarray(const container_type& c, const extents_type& ext,
       const Alloc& a);
template<class Alloc>
  constexpr mdarray(const container_type& c, const mapping_type& m,
       const Alloc& a);

template<class Alloc>
  constexpr mdarray(container_type&& c, const extents_type& ext, const
       Alloc& a);
template<class Alloc>
  constexpr mdarray(container_type&& c, const mapping_type& m, const
       Alloc& a);


template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class OtherContainer,
         class Alloc>
  explicit(see below)
  constexpr mdarray(
    const mdarray<OtherElementType, OtherExtents,
                  OtherLayoutPolicy, OtherContainer>& other, const
       Alloc& a);

template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class Accessor,
         class Alloc>
  explicit(see below)
  constexpr mdarray(const mdspan<OtherElementType, OtherExtents,
                                 OtherLayoutPolicy, Accessor>& other,
                    const Alloc& a);

constexpr mdarray& operator=(const mdarray& rhs) = default;
```

```
constexpr mdarray& operator=(mdarray&& rhs) = default;

// [mdarray.members], mdarray members
template<class... OtherIndexTypes>
  constexpr reference operator[](OtherIndexTypes... indices);
template<class OtherIndexType>
  constexpr reference operator[](span<OtherIndexType, rank()> indices);
template<class OtherIndexType>
  constexpr reference operator[](const array<OtherIndexType, rank()>&
      indices);
template<class... OtherIndexTypes>
  constexpr const_reference operator[](OtherIndexTypes... indices)
      const;
template<class OtherIndexType>
  constexpr const_reference operator[](span<OtherIndexType, rank()>
      indices) const;
template<class OtherIndexType>
  constexpr const_reference operator[](const array<OtherIndexType,
      rank()>& indices) const;

constexpr size_type size() const;
[[nodiscard]] constexpr bool empty() const noexcept;

friend constexpr void swap(mdarray& x, mdarray& y) noexcept;

constexpr const extents_type& extents() const { return map_.extents(); }
constexpr pointer data() { return ctr\_.data(); }
constexpr const_pointer data() const { return ctr\_.data(); }
constexpr const mapping_type& mapping() const { return map_; }

template<class OtherElementType, class OtherExtents,
         class OtherLayoutType, class OtherAccessorType>
constexpr operator mdspan () const;

template<class OtherAccessorType = default_accessor<element_type>>
  constexpr mdspan<element_type, extents_type, layout_type,
      OtherAccessorType>
    to_mdspan(const OtherAccessorType& a =
      default_accessor<element_type>());
template<class OtherAccessorType = default_accessor<const element_type>>
  constexpr mdspan<const element_type, extents_type, layout_type,
      OtherAccessorType>
    to_mdspan(const OtherAccessorType& a =
      default_accessor<const_element_type>()) const;

static constexpr bool is_always_unique() {
  return mapping_type::is_always_unique();
}
static constexpr bool is_always_exhaustive() {
  return mapping_type::is_always_exhaustive();
}
static constexpr bool is_always_strided() {
  return mapping_type::is_always_strided();
}

constexpr bool is_unique() const {
  return map_.is_unique();
```

```
  }
  constexpr bool is_exhaustive() const {
    return map_.is_exhaustive();
  }
  constexpr bool is_strided() const {
    return map_.is_strided();
  }
  constexpr index_type stride(size_t r) const {
    return map_.stride(r);
  }

private:
  container_type ctr_;
  mapping_type map_; // exposition only
};

template <class Container, class... Integrals>
requires((is_convertible_v<Integrals, size_t> && ...) && sizeof...
        (Integrals) > 0)
explicit mdarray(const Container&, Integrals...)
  -> mdarray<typename Container::value_type, dextents<size_t, sizeof...
        (Integrals)>, layout_right, Container>;

template<class Container, class Extents>
mdarray(const Container&, const Extents&)
  -> mdarray<typename Container::value_type, Extents, layout_right,
        Container>;

template<class Container, class Mapping>
mdarray(const Container&, const Mapping&)
  -> mdarray<typename Container::value_type, typename
        Mapping::extents_type, layout_right, Container>;

template <class Container, class... Integrals>
requires((is_convertible_v<Integrals, size_t> && ...) && sizeof...
        (Integrals) > 0)
explicit mdarray(Container&&, Integrals...)
  -> mdarray<typename Container::value_type, dextents<size_t, sizeof...
        (Integrals)>, layout_right, Container>;

template<class Container, class Extents>
mdarray(Container&&, const Extents&)
  -> mdarray<typename Container::value_type, Extents, layout_right,
        Container>;

template<class Container, class Mapping>
mdarray(Container&&, const Mapping&)
  -> mdarray<typename Container::value_type, typename
        Mapping::extents_type, layout_right, Container>;

template<class ElementType, class Extents, class Layout, class Accessor>
mdarray(const mdspan<ElementType, Extents, Layout, Accessor>&)
  -> mdarray<ElementType, Extents, Layout>;

template<class Container, class Extents, class Alloc>
mdarray(const Container&, const Extents&, const Alloc&)
```

```
  -> mdarray<typename Container::value_type, Extents, layout_right,
        Container>;

template<class Container, class Mapping, class Alloc>
mdarray(const Container&, const Mapping&, const Alloc&)
  -> mdarray<typename Container::value_type, typename
        Mapping::extents_type, layout_right, Container>;

template<class Container, class Extents, class Alloc>
mdarray(Container&&, const Extents&, const Alloc&)
  -> mdarray<typename Container::value_type, Extents, layout_right,
        Container>;

template<class Container, class Mapping, class Alloc>
mdarray(Container&&, const Mapping&, const Alloc&)
  -> mdarray<typename Container::value_type, typename
        Mapping::extents_type, layout_right, Container>;

template<class ElementType, class Extents, class Layout, class Accessor,
        class Alloc>
mdarray(const mdspan<ElementType, Extents, Layout, Accessor>&, const
        Alloc&)
  -> mdarray<ElementType, Extents, Layout>;
```

2 *Mandates:*

(2.1) — `ElementType` is a a complete object type that is neither an abstract class type nor an array type,

(2.2) — `Extents` is a specialization of `extents`, and

(2.3) — `is_same_v<ElementType, typename Container::value_type>` is `true`.

3 `LayoutPolicy` shall meet the layout mapping policy requirements [mdspan.layoutpolicy.reqmts], and

4 `Container` shall meet the requirements of contiguous container.

5 Each specialization `MDA` of `mdarray` models `copyable` and

(5.1) — `is_nothrow_move_constructible_v<MDA>` is `true` if `is_nothrow_move_constructible_v<Container>` is `true`,

(5.2) — `is_nothrow_move_assignable_v<MDA>` is `true` if `is_nothrow_move_assignable_v<Container>` is `true` , and

(5.3) — `is_nothrow_swappable_v<MDA>` is `true` if `is_nothrow_swappable_v<Container>` is `true`.

6 A specialization of `mdarray` is a trivially copyable type if its `container_type` and `mapping_type` are trivially copyable types.

24.6.□.2 Exposition only functions

```
template<class ValueType, class Index>
decltype(auto) just-value(Index, ValueType&& t) { return
        forward<ValueType&&>(t); }

template<class ValueType, size_t N>
array<ValueType, N>
value-to-array(const ValueType& t)
{
    return [&]<size_t ... Indices>(index_sequence<Indices...>) {
        return array<ValueType, N>{ just-value(Indices, t)... };
    }( make_index_sequence<N>() );
}
```

24.6.[illegible].2 `mdarray` constructors [mdarray.ctors]

```
template<class... OtherIndexTypes>
  explicit constexpr mdarray(OtherIndexTypes... exts);
```

1 *Constraints:*

(1.1) — `(is_convertible_v<OtherIndexTypes, index_type> && ...)` is `true`,

(1.2) — `(is_nothrow_constructible_v<index_type, OtherIndexTypes> && ...)` is `true`,

(1.3) — `is_constructible_v<extents_type, OtherIndexTypes...>` is `true`,

(1.4) — `is_constructible_v<mapping_type, extents_type>` is `true`, and

(1.5) — if `container_type` is not a specialization of `array`, `is_constructible_v<container_type, size_t>` is `true`.

2 *Preconditions:*

— [2.1] Let `map` be `mapping_type(extents_type(static_cast<index_type>(std::move(exts)...)))`, then

(2.2) — if `container_type` is a specialization of `array`, `map.required_span_size() <= size(container_type())` is `true`, otherwise

(2.3) — `map.required_span_size() <= size(container_type(map.required_span_size()))` is `true`.

3 *Effects:*

(3.1) — Direct-non-list-initializes *`map_`* with `extents_type(static_cast<index_type>(std::move(exts))...)`, and

(3.2) — if `is_constructible_v<container_type, size_t>` is `true`, direct-non-list-initializes `ctr_` with `container_type(`*`map_`*`.required_span_size())`, otherwise default constructs `ctr_`.

```
constexpr mdarray(const extents_type& ext);
```

4 *Constraints:*

(4.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(4.2) — if `container_type` is not a specialization of `array`, `is_constructible_v<container_type, size_t>` is `true`.

5 *Preconditions:*

(5.1) — Let `map` be `mapping_type(ext)`, then

(5.2) — if `container_type` is a specialization of `array`, `map.required_span_size() <= size(container_type())` is `true`, otherwise

(5.3) — `map.required_span_size() <= size(container_type(map.required_span_size()))` is `true`.

6 *Effects:*

(6.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(6.2) — if `is_constructible_v<container_type, size_t>` is `true`, direct-non-list-initializes `ctr_` with `container_type(`*`map_`*`.required_span_size())`, otherwise default constructs `ctr_`.

```
constexpr mdarray(const mapping_type& map);
```

7 *Constraints:* if `container_type` is not a specialization of `array`, `is_constructible_v<container_type, size_t>` is `true`.

8 *Preconditions:*

(8.1) — if `container_type` is a specialization of `array`, `map.required_span_size() <= size(container_type())` is `true`, otherwise

(8.2) — `map.required_span_size() <= size(container_type(map.required_span_size()))` is `true`.

9 *Effects:*

(9.1) — Direct-non-list-initializes *`map_`* with `map`, and

(9.2) — If `is_constructible_v<container_type, size_t>` is `true`, direct-non-list-initializes `ctr_` with `container_type(`*`map_`*`.required_span_size())`, otherwise default constructs `ctr_`.

```
constexpr mdarray(const extents_type& ext, const value_type& val);
```

10 *Constraints:*

(10.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(10.2) — if `container_type` is not a specialization of `array`, `is_constructible_v<container_type, size_t, value_type>` is `true`.

11 *Preconditions:*

(11.1) — Let `map` be `mapping_type(ext)`, then

(11.2) — if `container_type` is a specialization of `array`, `map.required_span_size() <= size(container_type())` is `true`, otherwise

(11.3) — `map.required_span_size() <= size(container_type(map.required_span_size()))` is `true`.

12 *Effects:*

(12.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(12.2) — if `is_constructible_v<container_type, size_t, value_type>` is `true`, direct-non-list-initializes `ctr_` with `container_type(`*`map_`*`.required_span_size(), val)`, otherwise direct-non-list-initializes `ctr_` with *`value-to-array`*`<element_type, size(declval<container_type>())>()`.

```
constexpr mdarray(const mapping_type& m, const value_type& val);
```

13 *Constraints:* if `container_type` is not a specialization of `array`, `is_constructible_v<container_type, size_t, value_type>` is `true`.

14 *Preconditions:*

(14.1) — if `container_type` is a specialization of `array`, `map.required_span_size() <= size(container_type())` is `true`, otherwise

(14.2) — `map.required_span_size() <= size(container_type(map.required_span_size(), val))` is `true`.

15 *Effects:*

(15.1) — Direct-non-list-initializes *`map_`* with `m`, and

(15.2) — if `is_constructible_v<container_type, size_t, value_type>` is `true`, direct-non-list-initializes `ctr_` with `container_type(`*`map_`*`.required_span_size(), val)`, otherwise direct-non-list-initializes `ctr_` with *`value-to-array`*`<element_type, size(declval<container_type>())>()`.

```
template<class... OtherIndexTypes>
  explicit constexpr mdarray(const container_type& c, OtherIndexTypes...
        exts);
```

16 *Constraints:*

(16.1) — `(is_convertible_v<OtherIndexTypes, index_type> && ...)` is `true`,

(16.2) — `(is_nothrow_constructible_v<index_type, OtherIndexTypes> && ...)` is `true`,

(16.3) — `is_constructible_v<extents_type, OtherIndexTypes...>` is `true`,

(16.4) — `is_constructible_v<mapping_type, extents_type>` is `true`, and

17 *Preconditions:* `c.size() >= mapping_type(extents_type(static_cast<index_type>(std::move(exts))...)).required_span_size()` is `true`.

18 *Effects:*

(18.1) — Direct-non-list-initializes *`map_`* with `extents_type(static_cast<index_type>(std::move(exts))...)`, and

(18.2) — Direct-non-list-initializes `ctr_` with `c`.

```
constexpr mdarray(const container_type& c, const extents_type& ext);
```

19 *Constraints:* `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

20 *Preconditions:* `c.size() >= mapping_type(ext).required_span_size()` is `true`.

21 *Effects:*

(21.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(21.2) — Direct-non-list-initializes `ctr_` with `c`.

```
constexpr mdarray(const container_type& c, const mapping_type& m);
```

22 *Preconditions:* `c.size() >= m.required_span_size()` is `true`.

23 *Effects:*

(23.1) — Direct-non-list-initializes *`map_`* with `m`, and

(23.2) — Direct-non-list-initializes `ctr_` with `c`.

```
template<class... OtherIndexTypes>
  explicit constexpr mdarray(container_type&& c, OtherIndexTypes... exts);
```

24 *Constraints:*

(24.1) — `(is_convertible_v<OtherIndexTypes, index_type> && ...)` is `true`,

(24.2) — `(is_nothrow_constructible_v<index_type, OtherIndexTypes> && ...)` is `true`,

(24.3) — `is_constructible_v<extents_type, static_cast<index_type>(OtherIndexTypes)...>` is `true`,

(24.4) — `is_constructible_v<mapping_type, extents_type>` is `true`, and

25 *Preconditions:* `c.size() >= mapping_type(extents_type(static_cast<index_type>(std::move(exts))...)).required_span_size()` is `true`.

26 *Effects:*

(26.1) — Direct-non-list-initializes *`map_`* with `extents_type(static_cast<index_type>(std::move(exts))...)`, and

(26.2) — Direct-non-list-initializes `ctr_` with `std::move(c)`.

```
constexpr mdarray(container_type&& c, const extents_type& ext);
```

27 *Constraints:* `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

28 *Preconditions:* `c.size() >= mapping_type(ext).required_span_size()` is `true`.

29 *Effects:*

(29.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(29.2) — Direct-non-list-initializes `ctr_` with `std::move(c)`.

```
constexpr mdarray(container_type&& c, const mapping_type& m);
```

30 *Preconditions:* `c.size() >= m.required_span_size()` is `true`.

31 *Effects:*

(31.1) — Direct-non-list-initializes *`map_`* with `m`, and

(31.2) — Direct-non-list-initializes `ctr_` with `std::move(c)`.

```
template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class OtherContainer>
  explicit(see below)
  constexpr mdarray(const mdarray<OtherElementType, OtherExtents,
                                  OtherLayoutPolicy, OtherContainer>&
        other);
```

32 *Mandates:*

(32.1) — `is_constructible_v<Container, const OtherContainer&>` is `true`, and

(32.2) — `is_constructible_v<extents_type, OtherExtents>` is `true`.

33 *Constraints:*

(33.1) — `is_constructible_v<mapping_type, const OtherLayoutPolicy::template mapping<OtherExtents>&>` is `true`, and

(33.2) — `is_constructible_v<container_type, OtherContainer>` is `true`.

34 *Preconditions:* For each rank index `r` of `extents_type`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is `true`.

35 *Effects:*

(35.1) — Direct-non-list-initializes `ctr_` with `other.ctr_`, and

(35.2) — Direct-non-list-initializes *`map_`* with `other.`*`map_`*.

36 *Remarks:* The expression inside `explicit` is:

```
!is_convertible_v<const typename OtherLayoutPolicy::mapping_type&,
      mapping_type> ||
!is_convertible_v<const OtherContainer&, Container>
```

```
template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class Accessor>
  explicit(see below)
  constexpr mdarray(const mdspan<OtherElementType, OtherExtents,
                                 OtherLayoutPolicy, Accessor>& other);
```

37 *Mandates:*

(37.1) — `is_constructible_v<extents_type, OtherExtents>` is `true`.

38 *Constraints:*

(38.1) — `is_constructible_v<value_type, Accessor::reference>` is `true`,

(38.2) — `is_assignable_v<Accessor::reference, value_type>` is `true`,

(38.3) — `is_default_constructible_v<value_type>` is `true`,

(38.4) — `is_constructible_v<mapping_type, const OtherLayoutPolicy::template mapping<OtherExtents>&>` is `true`, and

(38.5) — if `container_type` is not a specialization of `array`, `is_constructible_v<container_type, size_t>` is `true`.

39 *Preconditions:*

(39.1) — For each rank index `r` of `extents_type`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is `true`, and

(39.2) — if `container_type` is a specialization of `array`, then `container_type().size() >= other.mapping().required_span_size()`.

40 *Effects:*

(40.1) — Direct-non-list-initializes *`map_`* with `other.mapping()`;

(40.2) — If `is_constructible_v<container_type, size_t>` is `true`, direct-non-list-initializes `ctr_` with `container_type(other.mapping().required_span_size())`, otherwise default constructs *`ctr_`*; and

(40.3) — For all unique multidimensional indices `i...` in `other.extents()`, assigns `other[i...]` to `ctr_[`*`map_`*`(i...)]`.

[Note: Requiring default constructibility of `value_type` means that `ctr_` may first be constructed with its required span size, and then filled by iterating over all unique multidimensional indices `i...` in the `mdarray`'s domain. Alternately, `ctr_` may be constructed via `ranges::to`, if the elements of `other` can be viewed by a range. The intent is to permit `ranges::to` initialization of `ctr_` if possible, without requiring a particular iteration order (as the best-performing order can depend sensitively on the two layouts) or even requiring all `mdspan` to be iterable by a range.— *end note]*

41 *Remarks:* The expression inside `explicit` is:

```
  !is_convertible_v<const typename OtherLayoutPolicy::mapping_type&,
        mapping_type> ||
  !is_convertible_v<Accessor::reference, value_type>
```

24.6.[illegible].3 `mdarray` constructors with allocators [mdarray.ctors.alloc]

```
template<class Alloc>
  constexpr mdarray(const extents_type& ext, const Alloc& a);
```

1 *Constraints:*

(1.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(1.2) — `is_constructible_v<container_type, size_t, Alloc>` is `true`.

2 *Preconditions:*

(2.1) — Let `map` be `mapping_type(ext)`, then

(2.2) — if `container_type` is a specialization of `array`, `map.required_span_size() <= size(container_type())` is `true`, otherwise

(2.3) — `map.required_span_size() <= size(container_type(map.required_span_size(), a))` is `true`.

3 *Effects:*

(3.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(3.2) — Direct-non-list-initializes `ctr_` with *`map_`*`.required_span_size()` as the first argument and `a` as the second argument.

```
template<class Alloc>
  constexpr mdarray(const mapping_type& map, const Alloc& a);
```

4 *Constraints:* `is_constructible_v<container_type, size_t, Alloc>` is `true`.

5 *Preconditions:*

(5.1) — if `container_type` is a specialization of `array`, `map.required_span_size() <= size(container_type())` is `true`, otherwise

(5.2) — `map.required_span_size() <= size(container_type(map.required_span_size(), a))` is `true`.

6 *Effects:*

(6.1) — Direct-non-list-initializes *`map_`* with `map`, and

(6.2) — Direct-non-list-initializes `ctr_` with `map.required_span_size()` as the first argument and `a` as the second argument.

```
template<class Alloc>
constexpr mdarray(const extents_type& ext, const value_type& val, const
        Alloc& a);
```

7 *Constraints:*

(7.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(7.2) — if `container_type` is not a specialization of `array`, `is_constructible_v<container_type, size_t, value_type, Alloc>` is `true`.

8 *Preconditions:*

(8.1) — Let `map` be `mapping_type(ext)`, then

(8.2) — if `container_type` is a specialization of `array`, `map.required_span_size() <= size(container_type())` is `true`, otherwise

(8.3) — `map.required_span_size() <= size(container_type(map.required_span_size(), val, a))` is `true`.

9 *Effects:*

(9.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(9.2) — if `is_constructible_v<container_type, size_t, value_type>` is `true`, direct-non-list-initializes `ctr_` with `container_type(`*`map_`*`.required_span_size(), val)`, otherwise direct-non-list-initializes `ctr_` with *`value-to-array`*`<element_type, size(declval<container_type>())>()`.

```
template<class Alloc>
constexpr mdarray(const mapping_type& map, const value_type& val, const
        Alloc& a);
```

10 *Constraints:* if `container_type` is not a specialization of `array`, `is_constructible_v<container_type, size_t, value_type>` is `true`.

11 *Preconditions:*

(11.1) — if `container_type` is a specialization of `array`, `map.required_span_size() <= size(container_type())` is `true`, otherwise

(11.2) — `map.required_span_size() <= size(container_type(map.required_span_size(), val, a))` is `true`.

12 *Effects:*

(12.1) — Direct-non-list-initializes *`map_`* with `map`, and

(12.2) — if `is_constructible_v<container_type, size_t, value_type>` is `true`, direct-non-list-initializes `ctr_` with `container_type(`*`map_`*`.required_span_size(), val)`, otherwise direct-non-list-initializes `ctr_` with *`value-to-array`*`<element_type, size(declval<container_type>())>()`.

```
template<class Alloc>
  constexpr mdarray(const container_type& c, const extents_type& ext,
        const Alloc& a);
```

13 *Constraints:*

(13.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(13.2) — `is_constructible_v<container_type, container_type, Alloc>` is `true`.

14 *Preconditions:* `c.size() >= mapping_type(ext).required_span_size()` is `true`.

15 *Effects:*

(15.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(15.2) — Direct-non-list-initializes `ctr_` with `c` as the first argument and `a` as the second argument.

```
template<class Alloc>
  constexpr mdarray(const container_type& c, const mapping_type& m, const
        Alloc& a);
```

16 *Constraints:* `is_constructible_v<container_type, container_type, Alloc>` is `true`.

17 *Preconditions:* `c.size() >= m.required_span_size()` is `true`.

18 *Effects:*

(18.1) — Direct-non-list-initializes *`map_`* with `m`, and

(18.2) — Direct-non-list-initializes `ctr_` with `c` as the first argument and `a` as the second argument.

```
template<class Alloc>
  constexpr mdarray(container_type&& c, const extents_type& ext, const
        Alloc& a);
```

19 *Constraints:*

(19.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(19.2) — `is_constructible_v<container_type, container_type, Alloc>` is `true`.

20 *Preconditions:* `c.size() >= mapping_type(ext).required_span_size()` is `true`.

21 *Effects:*

(21.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(21.2) — Direct-non-list-initializes `ctr_` with `std::move(c)` as the first argument and `a` as the second argument.

```
template<class Alloc>
  constexpr mdarray(container_type&& c, const mapping_type& m, const
        Alloc& a);
```

22 *Constraints:* `is_constructible_v<container_type, container_type, Alloc>` is `true`.

23 *Preconditions:* `c.size() >= m.required_span_size()` is `true`.

24 *Effects:*

(24.1) — Direct-non-list-initializes *`map_`* with `m`, and

(24.2) — Direct-non-list-initializes `ctr_` with `std::move(c)` as the first argument and `a` as the second argument.

```
template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class OtherContainer, class Alloc>
  explicit(see below)
  constexpr mdarray(const mdarray<OtherElementType, OtherExtents,
                                  OtherLayoutPolicy, OtherContainer>&
        other,
                    const Alloc& a);
```

25 *Mandates:*

(25.1) — `is_constructible_v<Container, OtherContainer, Alloc>` is `true`, and

(25.2) — `is_constructible_v<extents_type, OtherExtents>` is `true`.

26 *Constraints:*

(26.1) — `is_constructible_v<mapping_type, const OtherLayoutPolicy::template mapping<OtherExtents>&>` is `true`, and

(26.2) — `is_constructible_v<container_type, OtherContainer, Alloc>` is `true`.

27 *Preconditions:* For each rank index `r` of `extents_type`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is `true`.

28 *Effects:*

(28.1) — Direct-non-list-initializes *map_* with `other.map_`, and

(28.2) — Direct-non-list-initializes `ctr_` with `other.ctr_` as the first argument and `a` as the second argument.

29 *Remarks:* The expression inside `explicit` is:

```
!is_convertible_v<const typename OtherLayoutPolicy::mapping_type&,
      mapping_type> ||
!is_convertible_v<const OtherContainer&, Container>
```

```
template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class Accessor,
         class Alloc>
  explicit(see below)
  constexpr mdarray(const mdspan<OtherElementType, OtherExtents,
                                 OtherLayoutPolicy, Accessor>& other,
                    const Alloc& a);
```

30 *Mandates:* `is_constructible_v<extents_type, OtherExtents>` is `true`.

31 *Constraints:*

(31.1) — `is_constructible_v<container_type, size_t, Alloc>` is `true`,

(31.2) — `is_constructible_v<value_type, Accessor::reference>` is `true`,

(31.3) — `is_assignable_v<Accessor::reference, value_type>` is `true`,

(31.4) — `is_default_constructible_v<value_type>` is `true`, and

(31.5) — `is_constructible_v<mapping_type, const OtherLayoutPolicy::template mapping<OtherExtents>&>` is `true`.

32 *Preconditions:*

(32.1) — For each rank index `r` of `extents_type`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is `true`, and

(32.2) — if `container_type` is a specialization of `array`, then `container_type().size() >= other.mapping().required_span_size()`.

33 *Effects:*

(33.1) — Direct-non-list-initializes *map_* with `extents_type(other.extents())`;

(33.2) — direct-non-list-initializes `ctr_` with `container_type(map_.required_span_size(), a)`; and

(33.3) — for all unique multidimensional indices `i...` in other.extents(), assigns `other[i...]` to `ctr_[map_(i...)]`.

[Note: For intent, please see Note on the `mdarray` constructor taking an `mdspan` with no allocator.— *end note]*

34 *Remarks:* The expression inside `explicit` is:

```
  !is_convertible_v<const typename OtherLayoutPolicy::mapping_type&,
        mapping_type> ||
  !is_convertible_v<Accessor::reference, value_type> ||
  !is_convertible_v<Alloc, container_type::allocator_type>
```

24.6.□.4 `mdarray` members [mdarray.members]

```
template<class... OtherIndexTypes>
  constexpr reference operator[](OtherIndexTypes... indices);
template<class... OtherIndexTypes>
  constexpr const_reference operator[](OtherIndexTypes... indices) const;
```

1 *Constraints:*

(1.1) — `(is_convertible_v<OtherIndexTypes, index_type> && ...)` is `true`,

(1.2) — `(is_nothrow_constructible_v<index_type, OtherIndexTypes> && ...)` is `true`, and

(1.3) — `sizeof...(OtherIndexTypes) == rank()` is `true`.

2 Let `I` be `extents_type::`*`index-cast`*`(std::move(indices))`.

3 *Preconditions:* `I` is a multidimensional index in `extents()`. *[Note:* This implies that *`map_`*`(I...) <`*`map_`*`.required_span_size()` is `true`.— *end note]*;

4 *Effects:* Equivalent to: `return` *`acc_`*`.access(`*`ptr_`*`,`*`map_`*`(static_cast<index_type>(std::move(indices))...));`

```
template<class OtherIndexType>
  constexpr reference operator[](span<OtherIndexType, rank()> indices);
template<class OtherIndexType>
  constexpr const_reference operator[](span<OtherIndexType, rank()>
        indices) const;
template<class OtherIndexType>
  constexpr reference operator[](const array<OtherIndexType, rank()>&
        indices);
template<class OtherIndexType>
  constexpr const_reference operator[](const array<OtherIndexType,
        rank()>& indices) const;
```

5 *Constraints:*

(5.1) — `is_convertible_v<const OtherIndexType&, index_type>` is `true`, and

(5.2) — `is_nothrow_constructible_v<index_type, const OtherIndexType&>` is `true`.

6 *Effects:* Let `P` be a parameter pack such that `is_same_v<make_index_sequence<rank()>, index_sequence<P...>>` is `true`.
Equivalent to: `return operator[](as_const(indices[P])...);`

```
constexpr size_type size() const noexcept;
```

7 *Precondition:* The size of the multidimensional index space `extents()` is a representable value of type `size_type` ([basic.fundamental]).

8 *Returns:* `extents().`*`fwd-prod-of-extents`*`(rank())`.

```
[[nodiscard]] constexpr bool empty() const noexcept;
```

9 *Returns:* `true` if the size of the multidimensional index space `extents()` is 0, otherwise `false`.

```
friend constexpr void swap(mdarray& x, mdarray& y) noexcept;
```

10 *Effects:* Equivalent to:

```
swap(x.ctr_, y.ctr_);
swap(x.map_, y.map_);
```

```
  template<class OtherElementType, class OtherExtents,
           class OtherLayoutType, class OtherAccessorType>
  operator mdspan ();
```

11 *Constraints:* `is_assignable_v<mdspan<element_type, extents_type, layout_type>, mdspan<OtherElementType, OtherExtents, OtherLayoutType, OtherAccessorType>>` is `true`.

12 *Returns:* `mdspan(data(),`*`map_`*`)`

```
  template<class OtherAccessorType>
    constexpr mdspan<element_type, extents_type, layout_type,
        OtherAccessorType>
      to_mdspan(const OtherAccessorType& a =
        default_accessor<element_type>());
```

13 *Constraints:* `is_assignable_v<mdspan<element_type, extents_type, layout_type>, mdspan<element_type, extents_type, layout_type, OtherAccessorType>>` is `true`.

14 *Returns:* `mdspan(data(),`*`map_`*`, a)`

```
  template<class OtherAccessorType>
    constexpr mdspan<const element_type, extents_type, layout_type,
        OtherAccessorType>
      to_mdspan(const OtherAccessorType& a = default_accessor<const
        element_type>()) const;
```

15 *Constraints:* `is_assignable_v<mdspan<const element_type, extents_type, layout_type>, mdspan<const element_type, extents_type, layout_type,`

`OtherAccessorType>>` is `true`.

16 *Returns:* `mdspan(data(),map_, a)`

Add to `mdspan` deduction guides

```
template<class ElementType, class Extents, class Layout, class Container>
mdspan(mdarray<ElementType, Extents, Layout, Container>) -> mdspan<
  decltype(declval<mdarray<ElementType, Extents, Layout, Container>>
        ().to_mdspan())::element_type,
  decltype(declval<mdarray<ElementType, Extents, Layout, Container>>
        ().to_mdspan())::extens_type,
  decltype(declval<mdarray<ElementType, Extents, Layout, Container>>
        ().to_mdspan())::layout_type,
  decltype(declval<mdarray<ElementType, Extents, Layout, Container>>
        ().to_mdspan())::accessor_type>;
```

§ 5 References

[P1684R0]
David Hollman, Christian Trott, Mark Hoemmen, Daniel Sundernland. 2019. mdarray: An Owning Multidimensional Array Analog of mdspan.
https://wg21.link/p1684r0